

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 license
 * Click nbfs://nbhost/SystemFileSystem/Templates/J2EE/EJB40/StatelessEjbClass.java to edit this
 template
 */
package ejb;
```

```
import entity.Dept;
import entity.Emp;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import java.util.Date;
import java.util.List;
```

```
/**
 *
 * @author DELL
 */
@Stateless
public class empdeptBean implements empdeptBeanLocal {
```

```
    @PersistenceContext(unitName = "my_empdept_unit")
```

```
    EntityManager em;
```

```
    @Override
```

```
    public void addDepartment(String name, String location) {
```

```
        // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody
```

```
        Dept d = new Dept();
```

```
        d.setDeptName(name);
```

```
        d.setLocation(location);

        em.persist(d);
    }
}
```

@Override

```
public List<Dept> getAllDepartment() {

    // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody

    return em.createQuery("SELECT d FROM Dept d", Dept.class).getResultList();
}
```

@Override

```
public void updateDepartment(String name, String location, int id) {

    // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody

    Dept d = em.find(Dept.class, id);
    if (d != null) {
        d.setDeptName(name);
        d.setLocation(location);
        em.merge(d);
    }
}
```

@Override

```
public void deleteDepartment(int id, String name, String location) {

    // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody

    Dept d = em.find(Dept.class, id);
    if (d != null) {
        em.remove(d);
    }
}
```

```
}
```

```
@Override
```

```
public void addEmployee(String ename, double salary, Dept deptid, Date dateofjoining) {  
    // throw new UnsupportedOperationException("Not supported yet."); // Generated from  
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody  
    Emp e = new Emp();  
    e.setName(ename);  
    e.setSalary(salary);  
    e.setDeptid(deptid);  
    e.setDateofjoining(dateofjoining);  
}
```

```
@Override
```

```
public List<Emp> getAllEmployees() {  
    // throw new UnsupportedOperationException("Not supported yet."); // Generated from  
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody  
    return em.createQuery("SELECT e FROM Emp e", Emp.class).getResultList();  
}
```

```
@Override
```

```
public void updateEmployee(int empno, String ename, double salary, Dept deptid, Date  
dateofjoining) {  
    // throw new UnsupportedOperationException("Not supported yet."); // Generated from  
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody  
    Emp e = em.find(Emp.class, empno);  
    if (e != null) {  
        e.setDateofjoining(dateofjoining);  
        e.setName(ename);  
        e.setSalary(salary);  
        e.setDeptid(deptid);  
        em.merge(e);  
    }  
}
```

```

    }
}

@Override

public void deleteEmployee(int empno, String ename, double salary, Dept deptid, Date
dateofjoining) {

    // throw new UnsupportedOperationException("Not supported yet."); // Generated from
nbfs://nbhost/SystemFileSystem/Templates/Classes/Code/GeneratedMethodBody

    Emp e = em.find(Emp.class, empno);

    if (e != null) {
        em.remove(e);
    }
}

}

}



---


/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
license
 * Click nbfs://nbhost/SystemFileSystem/Templates/J2EE/EJB40/SessionLocal.java to edit this
template
 */
package ejb;

import entity.Dept;
import entity.Emp;
import jakarta.ejb.Local;
import java.util.Date;
import java.util.List;

/**
 *
 * @author DELL

```

```

*/
@Local
public interface empdeptBeanLocal {

    //Department Operations
    void addDepartment(String name, String location);

    List<Dept> getAllDepartment();

    void updateDepartment(String name, String location, int id);

    void deleteDepartment(int id, String name, String location);

    //Employee Operations
    void addEmployee(String ename, double salary, Dept deptid, Date dateofjoining);

    List<Emp> getAllEmployees();

    void updateEmployee(int empno, String ename, double salary, Dept deptid, Date dateofjoining);

    void deleteEmployee(int empno, String ename, double salary, Dept deptid, Date dateofjoining);

}

```

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package entity;

import jakarta.persistence.Basic;

```

```

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import jakarta.xml.bind.annotation.XmlRootElement;
import jakarta.xml.bind.annotation.XmlTransient;
import java.io.Serializable;
import java.util.Collection;

/**
 *
 * @author DELL
 */
@Entity
@Table(name = "dept")
@XmlRootElement
@NamedQueries({
    @NamedQuery(name = "Dept.findAll", query = "SELECT d FROM Dept d"),
    @NamedQuery(name = "Dept.findByDeptid", query = "SELECT d FROM Dept d WHERE d.deptid = :deptid"),
    @NamedQuery(name = "Dept.findByDeptName", query = "SELECT d FROM Dept d WHERE d.deptName = :deptName"),
    @NamedQuery(name = "Dept.findByLocation", query = "SELECT d FROM Dept d WHERE d.location = :location")})

```

```

public class Dept implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "deptid")
    private Integer deptid;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 100)
    @Column(name = "dept_name")
    private String deptName;
    @Basic(optional = false)
    @NotNull
    @Size(min = 1, max = 100)
    @Column(name = "location")
    private String location;
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "deptid")
    private Collection<Emp> empCollection;

    public Dept() {
    }

    public Dept(Integer deptid) {
        this.deptid = deptid;
    }

    public Dept(Integer deptid, String deptName, String location) {
        this.deptid = deptid;
        this.deptName = deptName;
    }
}

```

```
    this.location = location;  
}
```

```
public Integer getDeptid() {  
    return deptid;  
}
```

```
public void setDeptid(Integer deptid) {  
    this.deptid = deptid;  
}
```

```
public String getDeptName() {  
    return deptName;  
}
```

```
public void setDeptName(String deptName) {  
    this.deptName = deptName;  
}
```

```
public String getLocation() {  
    return location;  
}
```

```
public void setLocation(String location) {  
    this.location = location;  
}
```

```
@XmlTransient
```

```
public Collection<Emp> getEmpCollection() {  
    return empCollection;  
}
```



```
public void setEmpCollection(Collection<Emp> empCollection) {  
    this.empCollection = empCollection;  
}
```

@Override

```
public int hashCode() {  
    int hash = 0;  
    hash += (deptid != null ? deptid.hashCode() : 0);  
    return hash;  
}
```

@Override

```
public boolean equals(Object object) {  
    // TODO: Warning - this method won't work in the case the id fields are not set  
    if (!(object instanceof Dept)) {  
        return false;  
    }  
    Dept other = (Dept) object;  
    if ((this.deptid == null && other.deptid != null) || (this.deptid != null &&  
!this.deptid.equals(other.deptid))) {  
        return false;  
    }  
    return true;  
}
```

@Override

```
public String toString() {  
    return "entity.Dept[ deptid=" + deptid + " ]";  
}
```

```
}
```

```
/*
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this  
license
```

```
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
```

```
*/
```

```
package entity;
```

```
import jakarta.persistence.Basic;
```

```
import jakarta.persistence.Column;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

```
import jakarta.persistence.JoinColumn;
```

```
import jakarta.persistence.ManyToOne;
```

```
import jakarta.persistence.NamedQueries;
```

```
import jakarta.persistence.NamedQuery;
```

```
import jakarta.persistence.Table;
```

```
import jakarta.persistence.Temporal;
```

```
import jakarta.persistence.TemporalType;
```

```
import jakarta.validation.constraints.NotNull;
```

```
import jakarta.validation.constraints.Size;
```

```
import jakarta.xml.bind.annotation.XmlRootElement;
```

```
import java.io.Serializable;
```

```
import java.util.Date;
```

```
/**
```

```
 *
```

```
 * @author DELL
```

```
*/
```

@Entity

@Table(name = "emp")

@XmlRootElement

@NamedQueries({

 @NamedQuery(name = "Emp.findAll", query = "SELECT e FROM Emp e"),

 @NamedQuery(name = "Emp.findByEmpno", query = "SELECT e FROM Emp e WHERE e.empno = :empno"),

 @NamedQuery(name = "Emp.findByEname", query = "SELECT e FROM Emp e WHERE e.ename = :ename"),

 @NamedQuery(name = "Emp.findByDateofjoining", query = "SELECT e FROM Emp e WHERE e.dateofjoining = :dateofjoining"),

 @NamedQuery(name = "Emp.findBySalary", query = "SELECT e FROM Emp e WHERE e.salary = :salary"))})

public class Emp implements Serializable {

 private static final long serialVersionUID = 1L;

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 @Basic(optional = false)

 @Column(name = "empno")

 private Integer empno;

 @Basic(optional = false)

 @NotNull

 @Size(min = 1, max = 100)

 @Column(name = "ename")

 private String ename;

 @Basic(optional = false)

 @NotNull

 @Column(name = "dateofjoining")

 @Temporal(TemporalType.DATE)

 private Date dateofjoining;

 // @Max(value=?) @Min(value=?)//if you know range of your decimal fields consider using these annotations to enforce field validation

```
@Basic(optional = false)
@NotNull
@Column(name = "salary")
private Double salary;

@JoinColumn(name = "deptid", referencedColumnName = "deptid")
@ManyToOne(optional = false)
private Dept deptid;

public Emp() {
}

public Emp(Integer empno) {
    this.empno = empno;
}

public Emp(Integer empno, String ename, Date dateofjoining, Dept deptid, Double salary) {
    this.empno = empno;
    this.ename = ename;
    this.dateofjoining = dateofjoining;
    this.salary = salary;
    this.deptid = deptid;
}

public Integer getEmpno() {
    return empno;
}

public void setEmpno(Integer empno) {
    this.empno = empno;
}
```

```
public String getName() {  
    return ename;  
}
```

```
public void setName(String ename) {  
    this.ename = ename;  
}
```

```
public Date getDateofjoining() {  
    return dateofjoining;  
}
```

```
public void setDateofjoining(Date dateofjoining) {  
    this.dateofjoining = dateofjoining;  
}
```

```
public Double getSalary() {  
    return salary;  
}
```

```
public void setSalary(Double salary) {  
    this.salary = salary;  
}
```

```
public Dept getDeptid() {  
    return deptid;  
}
```

```
public void setDeptid(Dept deptid) {  
    this.deptid = deptid;  
}
```

@Override

```
public int hashCode() {  
    int hash = 0;  
    hash += (empno != null ? empno.hashCode() : 0);  
    return hash;  
}
```

@Override

```
public boolean equals(Object object) {  
    // TODO: Warning - this method won't work in the case the id fields are not set  
    if (!(object instanceof Emp)) {  
        return false;  
    }  
    Emp other = (Emp) object;  
    if ((this.empno == null && other.empno != null) || (this.empno != null &&  
!this.empno.equals(other.empno))) {  
        return false;  
    }  
    return true;  
}
```

@Override

```
public String toString() {  
    return "entity.Emp[ empno=" + empno + " ]";  
}
```

```
}
```

/*

* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license

* Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Servlet.java to edit this template

*/

```
package servlets;
```

```
import ejb.empdeptBeanLocal;
```

```
import entity.Dept;
```

```
import jakarta.ejb.EJB;
```

```
import java.io.IOException;
```

```
import java.io.PrintWriter;
```

```
import jakarta.servlet.ServletException;
```

```
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
```

```
import jakarta.servlet.http.HttpServletResponse;
```

```
import java.text.SimpleDateFormat;
```

```
import java.util.Collection;
```

```
import java.util.Date;
```

```
/**
```

```
*
```

```
* @author DELL
```

```
*/
```

```
@WebServlet(name = "empdeptServlet", urlPatterns = {"/empdeptServlet"})
```

```
public class empdeptServlet extends HttpServlet {
```

```
    @EJB
```

```
    empdeptBeanLocal edp;
```

```
/**
```

```

* Processes requests for both HTTP <code>GET</code> and <code>POST</code>
* methods.
*
* @param request servlet request
* @param response servlet response
* @throws ServletException if a servlet-specific error occurs
* @throws IOException if an I/O error occurs
*/

```

```

protected void processRequest(HttpServletRequest request, HttpServletResponse response)

```

```

    throws ServletException, IOException {
response.setContentType("text/html;charset=UTF-8");
try (PrintWriter out = response.getWriter()) {
    /* TODO output your page here. You may use following sample code. */
    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet empdeptServlet</title>");
    out.println("</head>");
    out.println("<body>");

```

```

// edp.addDepartment("Information Technology", "Surat");
Dept dept = edp.getAllDepartment().get(0); // pick first department
SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
// Date doj = sdf.parse("13-06-2005");

```

```

// edp.addEmployee("Dhruvi", 67000, dept, doj);
edp.updateDepartment("Microbiology", "Rajkot", 2);

```

```

Collection<Dept> Dept = edp.getAllDepartment();
for (Dept d : Dept) {

```



```

        out.println("<br/>id: " + d.getDeptid() + "<br/>" + d.getDeptName() + "<br/>" +
d.getLocation());
    }

    out.println("<h1>Servlet empdeptServlet at " + request.getContextPath() + "</h1>");
    out.println("</body>");
    out.println("</html>");
}
}

```

// <editor-fold defaultstate="collapsed" desc="HttpServlet methods. Click on the + sign on the left to edit the code.">

```

/**
 * Handles the HTTP <code>GET</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs
 */
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Handles the HTTP <code>POST</code> method.
 *
 * @param request servlet request
 * @param response servlet response
 * @throws ServletException if a servlet-specific error occurs
 * @throws IOException if an I/O error occurs

```

```

*/
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

/**
 * Returns a short description of the servlet.
 *
 * @return a String containing servlet description
 */
@Override
public String getServletInfo() {
    return "Short description";
} // </editor-fold>
}

```